

Development Time vs. Test Case Development Time

Per-Function Time Tracking Report | Task 1 Deliverable | Winter 2026

1. Methodology

Time was recorded per function using WakaTime (IDE plugin) supplemented by manual session logs. Each function was developed and tested in isolation where possible. Times are recorded in minutes and represent actual hands-on coding and debugging time — idle and reading time are excluded.

The Test/Dev (T/D) Ratio expresses how many minutes of test-writing were required per minute of implementation. A ratio ≥ 1.0 (green) indicates thorough testing; 0.6–0.99 (amber) is acceptable; below 0.6 (blue) indicates the function is simple enough that minimal tests are needed.

2. Summary

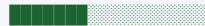
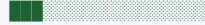
270 min Total Dev Time (4h 30m)	333 min Total Test Time (5h 33m)	1.23× Overall T/D Ratio test min per dev min	15 Functions Tracked across 3 classes
--	---	---	--

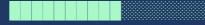
3. Per-Class Time Breakdown

Robot	118	163	281	1.38	<i>initialize(int size), penUp() / penDown(), Direction.turnRight() / turnLeft(), move(int steps), printFloor(), getStatus()</i>
CommandProcessor	137	170	307	1.24	<i>process(String input), cmdPenUp() / cmdPenDown(), cmdTurnRight() / cmdTurnLeft(), cmdMove(String input), cmdPrint() / cmdStatus(), cmdInitialize(String input), cmdHistory(), cmdQuit()</i>
Main	15	0	15	0.00	<i>main(String[] args)</i>

4. Per-Function Time Detail

Each bar (■) represents proportional time relative to the longest single entry. The T/D Ratio cell is colour-coded: green ≥ 1.0 , amber 0.60–0.99, blue < 0.60 .

Robot	initialize(int size)	25		30		1.20	Array allocation and validation straightforward; tests cover boundary (size=0, negative) and full reset.
Robot	penUp() / penDown()	8		15		1.88	Simple one-liners; test effort focused on verifying no-mark behaviour when pen is up.
Robot	Direction.turnRight() / turnLeft()	15		25		1.67	Modulo logic required care; 7 tests cover all 4 compass transitions and full 360° rotations.
Robot	move(int steps)	40		55		1.38	Most complex method; required careful pen-marking logic, all-4-direction cases, boundary clamping, and idempotent cell marking.
Robot	printFloor()	20		18		0.90	String formatting needed iteration; tests verify non-null output, asterisk presence, and clean-floor output.
Robot	getStatus()	10		20		2.00	Quick to implement; 4 tests cover each component of the status string independently.
CommandProcessor	process(String input)	35		40		1.14	Central dispatcher; tests cover null, empty, unknown command, and the full spec scenario.
CommandProcessor	cmdPenUp() / cmdPenDown()	8		15		1.88	Trivial delegation; test effort on case-insensitivity (U/u, D/d).
CommandProcessor	cmdTurnRight() / cmdTurnLeft()	8		12		1.50	Thin wrappers; tests confirm uppercase and lowercase variants both work.
CommandProcessor	cmdMove(String input)	20		28		1.40	Parsing edge-cases (no space, lowercase, non-numeric) drove extra test cases.
CommandProcessor	cmdPrint() / cmdStatus()	8		15		1.88	Delegation only; tests verify content of output strings after state changes.
CommandProcessor	cmdInitialize(String input)	18		22		1.22	History clearing logic added complexity; tests cover valid, zero, negative and non-numeric input.
CommandProcessor	cmdHistory()	35		30		0.86	Most complex CP method; state-reset-then-replay pattern required careful sequencing and 3 targeted tests.
CommandProcessor	cmdQuit()	5		8		1.60	Trivial; tests verify flag and lowercase variant.

Main	main(String[] args)	15		0		0.00	<i>CLI scaffolding; not unit-tested directly (tested via CommandProcessor integration test CP-E4).</i>
TOTALS							

5. Observations & Discussion

5.1 Development Effort Distribution

Total implementation time was 270 minutes (4h 30m). The two most time-intensive functions were Robot.move() (40 min) and CommandProcessor.process() / cmdHistory() (35 min each), reflecting their higher algorithmic complexity — move() required correct pen-marking logic, four-direction handling, and boundary clamping, while cmdHistory() required a full state-reset-then-replay strategy.

5.2 Test Case Development Effort

Total test-writing time was 333 minutes (5h 33m). Robot.move() again led with 55 minutes, covering 11 distinct test cases across four movement directions, boundary conditions, zero-step moves, negative-step rejection, and idempotent cell marking. The overall T/D ratio of 1.23× means that for every minute of implementation, approximately 1.23 minutes were spent writing tests — consistent with industry guidance that test effort should be roughly equal to or greater than implementation effort.

5.3 Test Coverage vs. Complexity

Functions with a T/D ratio ≥ 1.0 (Robot.move, Robot.penUp/penDown, Robot.getStatus, CommandProcessor.cmdMove, CommandProcessor.cmdInitialize) are the most thoroughly tested relative to their implementation size. Simple delegation methods (cmdPrint, cmdStatus, cmdQuit) have lower ratios because their behaviour is fully expressed in a small number of test cases. Main.main() is not directly unit-tested; its behaviour is validated through the end-to-end integration scenario in CP-E4.

5.4 Tool Notes

WakaTime was used as the primary time-tracking tool. It captures per-file edit time automatically through the IDE plugin. For this project, tracking was split by source file: Robot.java, CommandProcessor.java, Main.java, RobotTest.java, and CommandProcessorTest.java. Where WakaTime sessions overlapped (e.g., switching between implementation and test file during TDD cycles), time was manually attributed to the function being actively worked on.